P2953R2
Forbid defaulting operator=(X&&) &&

Published Proposal, 2025-01-08

Author:
Arthur O'Dwyer

Audience:
SG17

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Abstract

Current C++ permits explicitly-defaulted special members to differ from their implicitly-defaulted counterparts in various ways, including parameter type and ref-qualification. This permits implausible signatures like `A& operator=(const A&) && = default`, where the left-hand operand is rvalue-ref-qualified. We propose to forbid such signatures.

Table of Contents

§ 1. Changelog

- R2 (pre-Hagenberg):
  - Add poll results from the EWG telecon of 2025-01-08.
- R1:
  - Propose an "ambitious" overhaul as well as the "conservative" surgery.

§ 2. Motivation and proposal

Currently, [dcl.fct.def.default]/2.5 permits an explicitly defaulted special member function to differ from the implicit one by adding *ref-qualifiers*, but not *cv-qualifiers*.

For example, the signature `const A& operator=(const A&) const& = default` is forbidden because it is additionally const-qualified, and also because its return type differs from the implicitly-defaulted `A&`. This might be considered unfortunate, because that's a reasonable signature for a const-assignable proxy-reference type. But programmers aren't clamoring for that signature to be supported, so we do not propose it here.

Our concern is that the *unrealistic* signature `A& operator=(const A&) && = default` is *permitted!* This has three minor drawbacks:

- The possibility of these unrealistic signatures makes C++ harder to understand. Before writing [P2952], Arthur didn't know such signatures were possible.
- The wording to permit these signatures is at least a tiny bit more complicated than if they weren't permitted.
- The quirky interaction with [CWG2586] and [P2952] discussed in the next subsection.

To eliminate all three drawbacks, we propose that a defaulted copy/move assignment operator should not be allowed to add to its implicit signature an *rvalue* ref-qualifier.

```
struct C {
  C& operator=(const C&) && = default;
    // C++26: Well-formed
    // Proposed: Unusable (deleted or ill-formed)
};

struct D {
  D& operator=(this D&& self, const C&) = default;
    // C++26: Well-formed
    // Proposed: Unusable (deleted or ill-formed)
};
```

This proposal applies only to *explicitly defaulted* `operator=`, and only when the object parameter itself is rvalue-ref-qualified. We propose that it remain legal to write rvalue-ref-qualified `operator=` functions by hand; the compiler simply shouldn't assume it knows how to default one.

§ 2.1. Interaction with P2952

[CWG2586] (adopted for C++23) permits `operator=` to have an explicit object parameter.

[P2952] (currently in CWG for C++26) proposes that `operator=` should (also) be allowed to have a placeholder return type. If C++26 gets P2952 without P2953, then we'll have:

```
struct C {
  auto&& operator=(this C&& self, const C&) { return self; }
    // C++26: OK, still deduces C&&

  auto&& operator=(this C&& self, const C&) = default;
    // C++23: Ill-formed, return type contains auto
    // C++26 after P2952: OK, deduces C&
    // Proposed (conservative): Deleted, object parameter is not C&
    // Proposed (ambitious): Ill-formed, object parameter is not C&
};
```

The first, non-defaulted, operator "does the natural thing" by returning its left-hand operand, and deduces `C&&`. The second operator also "does the natural thing" by being defaulted; but it deduces `C&`. (For rationale, see [P2952] §3.3 "Deducing `this` and CWG2586.") The two "natural" implementations deduce different types! This looks inconsistent.

If we adopt P2953 alongside P2952, then the second `operator=` will go back to being unusable, which reduces the perception of inconsistency.

	C++23	P2952
C++23	`C&&`/ill-formed	`C&&`/`C&`
P2953	`C&&`/ill-formed	`C&&`/deleted

§ 2.2. "Deleted" versus "ill-formed"

(See also [P2952] §3.2 "Defaulted as deleted".)

[dcl.fct.def.default]/2.6 goes out of its way to make many explicitly defaulted constructors, assignment operators, and comparison operators "defaulted as deleted," rather than ill-formed. This was done by [P0641] (resolving [CWG1331]), in order to support class templates with "canonically spelled" defaulted declarations:

```
struct A {
  // Permitted by (2.4)
  A(A&) = default;
  A& operator=(A&) = default;
};

template<class T>
struct C {
  T t_;
  explicit C();
  // Permitted, but defaulted-as-deleted, by (2.6), since P0641
  C(const C&) = default;
  C& operator=(const C&) = default;
};

C<A> ca; // OK
```

There is similar wording in [class.spaceship] and [class.eq]. We don't want to interfere with these use-cases; that is, we want to continue permitting programmers to write things like the above `C<A>`.

§ 2.3. Existing corner cases

There is vendor divergence in some corner cases. Here is a table of the divergences we found, plus our opinion as to the conforming behavior, and our proposed behavior.

URL	Code	Clang	GCC	MSVC	EDG	Correct	Proposed (conservative)
link	`C& operator=(C&) = default;`	✓	✓	✓	✓	✓	✓
link	`C& operator=(const C&&) = default;`	deleted	✗	✗	deleted	deleted	deleted
link	`C& operator=(const C&) const = default;`	deleted	✗	✗	deleted	deleted	deleted
link	`C& operator=(const C&) && = default;`	✓	✓	✓	✓	✓	deleted
link	`C&& operator=(const C&) && = default;`	✗	✗	✗	✗	✗	✗
link	`template<class> struct C { static const C& f(); C& operator=(decltype(f())) = default; };`	✓	✗	✗	✗	✓	✓
link	`struct M { M& operator=(const M&) volatile; }; struct C { volatile M m; C& operator=(const C&) = default; };`	deleted	deleted	deleted	inconsistent	deleted	deleted

§ 3. Implementation experience

Arthur has implemented § 5 Proposed wording (conservative) in his fork of Clang, and used it to compile both LLVM/Clang/libc++ and another large C++17 codebase. Naturally, it caused no problems except in the relevant parts of Clang's own test suite.

There is no implementation experience for § 6 Proposed wording (ambitious).

§ 4. Straw poll results

Arthur O'Dwyer presented P2592R1 (not P2953, but P2952) in the EWG telecon of 2025-01-08. In addition to the vote forwarding P2952 to CWG, the following straw poll relevant to P2953 was taken. The result was interpreted as "no consensus"; but the numbers (6 for, 1 against) are still a strong signal that EWG was favorably inclined toward P2953 in general.

	SF	*F*	*N*	*A*	*SA*
EWG prefers this paper contains the change in P2953 (banning explicitly defaulted operator= with rvalue ref-qualifier). [Chair: This means EWG wants to see this paper again.]	*2*	*4*	*9*	*1*	*0*

§ 4.1. Proposed polls

The author suggests the following straw polls.

	SF	*F*	*N*	*A*	*SA*
EWG prefers the approach in § 6 Proposed wording (ambitious), which additionally makes `S& operator=(const S&&) = default;` *and* `C& operator=(const C&) const = default;` *ill-formed rather than merely defaulted-as-deleted.*	–	—	—	—	—
Advance P2953 to CWG for C++26.	–	—	—	—	—

§ 5. Proposed wording (conservative)

§ 5.1. [dcl.fct.def.default]

DRAFTING NOTE: The only defaultable special member functions are default constructors, copy/move constructors, copy/move assignment operators, and destructors. Of these, only the assignment operators can ever be cvref-qualified at all.

Modify [dcl.fct.def.default] as follows:

1. A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall

- (1.1) be a special member function or a comparison operator function ([over.binary]), and
- (1.2) not have default arguments.

2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:

- (2.1) if F_2 is an assignment operator, ~~F_1 and F_2 may have differing *ref-qualifiers*~~ F_1 may have an lvalue *ref-qualifier*;
- (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ is an assignment operator with an implicit object parameter of type `C&`, F_1 may ~~be an explicit object member function whose~~ have an explicit object parameter ~~is~~ of type ~~(possibly different) "reference to C"~~ `C&`, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
- (2.3) F_1 and F_2 may have differing exception specifications; and
- (2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type `C&`.

If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then:

- (2.5) if ~~F_1~~ F_2 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1's non-object parameter type is not a reference, the program is ill-formed;
- (2.6) otherwise, if F_1 is explicitly defaulted on its first declaration, it is defined as deleted;
- (2.7) otherwise, the program is ill-formed.

[...]

§ 6. Proposed wording (ambitious)

DRAFTING NOTE: The intent of this "ambitious" wording is to lock down the signatures of defaultable member functions as much as possible, and make errors as eager as possible, *except* in the cases covered by § 2.2 "Deleted" versus "ill-formed" (which we want to keep working, i.e., "defaulted as deleted").

§ 6.1. [dcl.fct.def.default]

> **DRAFTING NOTE:** Basically all of this wording is concerned specifically with copy/move assignment operators, so it might be nice to move it out of [dcl.fct.def.default] and into [class.copy.assign]. Also note that right now a difference in `noexcept`-ness is handled explicitly by [dcl.fct.def.default] for special member functions but only by omission-and-note in [class.compare] for comparison operators.

Modify [dcl.fct.def.default] as follows:

> 1. A function definition whose *function-body* is of the form `= default ;` is called an *explicitly-defaulted* definition. A function that is explicitly defaulted shall
>
> - (1.1) be a special member function or a comparison operator function ([over.binary]), and
> - (1.2) not have default arguments.
>
> 2. An explicitly defaulted special member function F_1 is allowed to differ from the corresponding special member function F_2 that would have been implicitly declared, as follows:
>
> - (2.1) if F_2 is an assignment operator, ~~F_1 and F_2 may have differing *ref-qualifiers*~~ F_1 may have an lvalue *ref-qualifier*;
> - (2.2) if F_2 ~~has an implicit object parameter of type "reference to C"~~ is an assignment operator with an implicit object parameter of type `C&`, F_1 may ~~be an explicit object member function whose~~ have an explicit object parameter ~~is~~ of type ~~(possibly different) "reference to C"~~ `C&`, in which case the type of F_1 would differ from the type of F_2 in that the type of F_1 has an additional parameter;
> - (2.3) F_1 and F_2 may have differing exception specifications; ~~and~~
> - (2.4) if F_2 has a non-object parameter of type `const C&`, the corresponding non-object parameter of F_1 may be of type `C&`; and
> - (2.5) if F_2 has a non-object parameter of type `C&`, the corresponding non-object parameter of F_1 may be of type `const C&`.
>
> If the type of F_1 differs from the type of F_2 in a way other than as allowed by the preceding rules, then~~:~~
>
> - ~~(2.5) if F_1 is an assignment operator, and the return type of F_1 differs from the return type of F_2 or F_1's non-object parameter type is not a reference, the program is ill-formed;~~
> - ~~(2.6) otherwise, if F_1 is explicitly defaulted on its first declaration, it is defined as deleted;~~
> - ~~(2.7) otherwise,~~ the program is ill-formed.
>
> [...]

§ References

§ Informative References

[CWG1331]
Daniel Krügler. *const mismatch with defaulted copy constructor*. June 2011. URL: https://cplusplus.github.io/CWG/issues/1331.html

[CWG2586]
Barry Revzin. *Explicit object parameter for assignment and comparison*. May–July 2022. URL: https://cplusplus.github.io/CWG/issues/2586.html

[P0641]
Daniel Krügler; Botond Ballo. *Resolving CWG1331: const mismatch with defaulted copy constructor*. November 2017. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0641r2.html

[P2952]
Arthur O'Dwyer; Matthew Taylor. *auto& operator=(X&&) = default*. August 2023. URL: https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2952r0.html